Code 1: Data Loading and Preparation

Examination Result Analytics

Step 1: Load the CSV dataset

```
data <- read.csv(file.choose())
```

Step 2: Display dataset dimensions

```
cat("DATASET DIMENSIONS\n")
print(dim(data))
```

```
Output:
DATASET DIMENSIONS
[1] 60 10
```

Step 3: Display column names

```
cat("\nCOLUMN NAMES\n")
print(names(data))
```

```
Output:
 COLUMN NAMES
 [1] "Roll_No"    "Section"    "Maths"      "Physics"    "Programming"
 [6] "English"    "Attendance" "Total"      "Percentage" "Grade"
```

Step 4: Display first six records

```
cat("\nFIRST SIX RECORDS\n")
print(head(data))
```

```
Output:
FIRST SIX RECORDS
  Roll_No Section Maths Physics Programming English Attendance Total
1     101       A  36.5    35.0        35.6    47.7       60.3 154.8
2     102       A  54.5    61.4        44.6    59.0       63.6 219.5
3     103       A  66.2    67.1        87.3    78.8       78.9 299.4
4     104       B  72.0    71.5        56.1    85.1       72.5 284.7
5     105       B  76.4    60.4        61.2    74.4       66.7 272.4
6     106       B  51.9    89.7        67.8    74.4       91.4 283.8
  Percentage  Grade
1      38.70   Pass
2      54.88 Second
3      74.85  First
4      71.18  First
5      68.10  First
6      70.95  First
```

Step 5: Display dataset structure

```
cat("\nDATA STRUCTURE\n")
str(data)
```

```
Output:
'data.frame':   60 obs. of  10 variables:
 $ Roll_No    : int  101 102 103 104 105 106 107 108 109 110 ...
 $ Section    : chr  "A" "A" "A" "B" ...
 $ Maths      : num  36.5 54.5 66.2 72 76.4 51.9 92.2 76.8 35 62.2 ...
 $ Physics    : num  35 61.4 67.1 71.5 60.4 89.7 69.5 51.5 66 57.6 ...
 $ Programming: num  35.6 44.6 87.3 56.1 61.2 67.8 55.3 85.1 65 58.1 ...
 $ English    : num  47.7 59 78.8 85.1 74.4 74.4 82.7 61.1 64 73.6 ...
 $ Attendance : num  60.3 63.6 78.9 72.5 66.7 91.4 87.1 87 75 73.4 ...
 $ Total      : num  155 220 299 285 272 ...
 $ Percentage : num  38.7 54.9 74.8 71.2 68.1 ...
 $ Grade      : chr  "Pass" "Second" "First" "First" ...
```

Step 6: Define subject columns

```
subjects <- c("Maths", "Physics", "Programming", "English")
```

Step 7: Standardize section names

```
data$Section <- toupper(trimws(data$Section))
```

Step 8: Calculate total marks

```
data$Total <- rowSums(data[, subjects], na.rm = FALSE)
```

Step 9: Calculate percentage

```
data$Percentage <- data$Total / length(subjects)
```

Step 10: Assign grades

```
data$Grade <- cut(
  data$Percentage,
  breaks = c(-Inf, 50, 60, 75, Inf),
  labels = c("Pass", "Second", "First", "Distinction"),
```

```
  right = FALSE
)
```

Step 11: Display prepared dataset

```
cat("\nPREPARED DATA - FIRST SIX RECORDS\n")
print(head(data))
```

```
Output:
PREPARED DATA - FIRST SIX RECORDS
  Roll_No Section Maths Physics Programming English Attendance Total
1     101       A  36.5    35.0        35.6    47.7       60.3 154.8
2     102       A  54.5    61.4        44.6    59.0       63.6 219.5
3     103       A  66.2    67.1        87.3    78.8       78.9 299.4
4     104       B  72.0    71.5        56.1    85.1       72.5 284.7
5     105       B  76.4    60.4        61.2    74.4       66.7 272.4
6     106       B  51.9    89.7        67.8    74.4       91.4 283.8
  Percentage  Grade
1     38.700   Pass
2     54.875 Second
3     74.850  First
4     71.175  First
5     68.100  First
6     70.950  First
```

Step 12: Display summary statistics

```
cat("\nPREPARED DATA SUMMARY\n")
print(summary(data))
```

```
Output:
PREPARED DATA SUMMARY
    Roll_No        Section             Maths          Physics     
 Min.   :101.0   Length:60          Min.   :35.00   Min.   :35.00  
 1st Qu.:115.8   N.unique : 2       1st Qu.:54.05   1st Qu.:60.08  
 Median :130.5   N.blank  : 0       Median :67.70   Median :66.85  
 Mean   :130.5   Min.nchar: 1       Mean   :66.49   Mean   :66.78  
 3rd Qu.:145.2   Max.nchar: 1       3rd Qu.:76.92   3rd Qu.:74.53  
 Max.   :160.0                      Max.   :94.70   Max.   :94.70  
  Programming        English        Attendance        Total      
 Min.   :35.00   Min.   :37.60   Min.   :60.30   Min.   :154.8  
 1st Qu.:53.83   1st Qu.:61.65   1st Qu.:73.30   1st Qu.:251.2  
 Median :60.35   Median :73.00   Median :77.95   Median :270.8  
 Mean   :62.14   Mean   :70.94   Mean   :79.20   Mean   :266.3  
 3rd Qu.:71.00   3rd Qu.:79.83   3rd Qu.:86.33   3rd Qu.:292.3  
 Max.   :94.70   Max.   :93.90   Max.   :99.30   Max.   :315.8  
   Percentage          Grade    
 Min.   :38.70   Pass       : 3  
 1st Qu.:62.80   Second     : 7  
 Median :67.70   First      :46  
 Mean   :66.59   Distinction: 4  
 3rd Qu.:73.08                   
 Max.   :78.95                   
```

Step 13: Save prepared dataset

```
write.csv(
  data,
  "prepared_examination_results.csv",
  row.names = FALSE
)
```

Step 14: Completion message

```
cat("\nDATA PREPARATION COMPLETED SUCCESSFULLY\n")
```

Code 2: Data Quality Assessment

Examination Result Analytics

Step 1: Load the prepared dataset

```
data <- read.csv(file.choose())
```

Step 2: Display dataset dimensions

```
cat("DATASET DIMENSIONS\n")
print(dim(data))
```

```
Output:
DATASET DIMENSIONS
[1] 60 10
```

Step 3: Check missing values in each column

```
cat("\nMISSING VALUES IN EACH COLUMN\n")
print(colSums(is.na(data)))
```

```
Output:
MISSING VALUES IN EACH COLUMN
    Roll_No     Section       Maths     Physics Programming     English
          0           0           0           0           0           0
 Attendance       Total  Percentage       Grade
          0           0           0           0
```

Step 4: Calculate total missing values

```
cat("\nTOTAL MISSING VALUES\n")
print(sum(is.na(data)))
```

```
Output:
TOTAL MISSING VALUES
[1] 0
```

Step 5: Check duplicate records

```
cat("\nNUMBER OF DUPLICATE ROWS\n")
print(sum(duplicated(data)))
```

```
Output:
NUMBER OF DUPLICATE ROWS
[1] 0
```

Step 6: Check section labels

```
cat("\nSECTION-WISE STUDENT COUNT\n")
print(table(data$Section, useNA = "ifany"))
```

```
Output:
SECTION-WISE STUDENT COUNT

 A  B
```

(Rest of Step 6 output and the Step 7 code were not visible in the screenshots.)

Step 7: (output only, as visible)

```
Output:
Maths : 0
Physics : 0
Programming : 0
English : 0
```

Step 8: Check invalid attendance

```
cat("\nINVALID ATTENDANCE VALUES\n")
print(sum(
  data$Attendance < 0 |
    data$Attendance > 100,
  na.rm = TRUE
))
```

```
Output:
INVALID ATTENDANCE VALUES
[1] 0
```

Step 9: Check missing values in subject marks

```
cat("\nMISSING SUBJECT MARKS\n")
for (subject in subjects) {
  cat(subject, ":", sum(is.na(data[[subject]])), "\n")
}
```

```
Output:
MISSING SUBJECT MARKS
Maths : 0
```

```
Physics : 0
Programming : 0
English : 0
```

Step 10: Display quality assessment summary

```
cat("\nDATA QUALITY ASSESSMENT SUMMARY\n")
cat("Total Records:", nrow(data), "\n")
cat("Total Columns:", ncol(data), "\n")
cat("Missing Values:", sum(is.na(data)), "\n")
cat("Duplicate Rows:", sum(duplicated(data)), "\n")
cat("\nDATA QUALITY ASSESSMENT COMPLETED\n")
```

```
Output:
DATA QUALITY ASSESSMENT SUMMARY
Total Records: 60
Total Columns: 10
Missing Values: 0
Duplicate Rows: 0

DATA QUALITY ASSESSMENT COMPLETED
```

Code 3: Descriptive Statistical Analysis

Examination Result Analytics

Step 1: Load the prepared dataset

```
data <- read.csv(file.choose())
```

Step 2: Define subject columns

```
subjects <- c("Maths", "Physics", "Programming", "English")
```

Step 3: Check subject columns

```
cat("SUBJECT COLUMNS\n")
print(subjects)
```

```
Output:
SUBJECT COLUMNS
[1] "Maths"       "Physics"     "Programming" "English"
```

Step 4: Calculate overall subject statistics

```
cat("\nOVERALL SUBJECT STATISTICS\n")

subject_statistics <- data.frame(
  Subject = subjects,
  Mean = sapply(data[subjects], mean, na.rm = TRUE),
  Median = sapply(data[subjects], median, na.rm = TRUE),
  Minimum = sapply(data[subjects], min, na.rm = TRUE),
  Maximum = sapply(data[subjects], max, na.rm = TRUE),
  Standard_Deviation = sapply(data[subjects], sd, na.rm = TRUE)
)

# Round only numeric columns
subject_statistics[, -1] <- round(subject_statistics[, -1], 2)

print(subject_statistics)
```

```
Output:
OVERALL SUBJECT STATISTICS
                  Subject  Mean Median Minimum Maximum Standard_Deviation
Maths               Maths 66.49  67.70    35.0    94.7              15.59
Physics           Physics 66.78  66.85    35.0    94.7              12.65
Programming Programming 62.14  60.35    35.0    94.7              14.03
English           English 70.94  73.00    37.6    93.9              12.95
```

Step 5: Overall percentage summary

```
cat("\nOVERALL PERCENTAGE SUMMARY\n")
print(summary(data$Percentage))
```

```
Output:
OVERALL PERCENTAGE SUMMARY
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  38.70   62.80   67.70   66.59   73.08   78.95
```

Step 6: Calculate overall average percentage

```
cat("\nOVERALL AVERAGE PERCENTAGE\n")
print(round(mean(data$Percentage, na.rm = TRUE), 2))
```

```
Output:
OVERALL AVERAGE PERCENTAGE
[1] 66.59
```

Step 7: Section-wise average percentage

```
cat("\nSECTION-WISE AVERAGE PERCENTAGE\n")

section_performance <- aggregate(
  Percentage ~ Section,
  data = data,
  FUN = mean,
  na.action = na.omit
)

section_performance$Percentage <- round(
  section_performance$Percentage, 2
)

print(section_performance)
```

```
Output:
SECTION-WISE AVERAGE PERCENTAGE
  Section Percentage
1       A      65.98
```

```
2       B      67.19
```

Step 8: Section-wise subject averages

```
cat("\nSECTION-WISE SUBJECT AVERAGES\n")

section_subject_averages <- aggregate(
  data[subjects],
  by = list(Section = data$Section),
  FUN = mean,
  na.action = na.omit
)

print(section_subject_averages)
```

```
Output:
  Section Maths Physics Programming English
1       A 66.97   66.28       59.72   70.94
2       B 66.01   67.27       64.55   70.95
```

Step 9: Find highest and lowest average subjects

```
subject_means <- colMeans(data[subjects], na.rm = TRUE)

cat("\nHIGHEST AVERAGE SUBJECT\n")
cat("Subject:", names(which.max(subject_means)), "\n")
cat("Average:", round(max(subject_means), 2), "\n")

cat("\nLOWEST AVERAGE SUBJECT\n")
cat("Subject:", names(which.min(subject_means)), "\n")
cat("Average:", round(min(subject_means), 2), "\n")
```

```
Output:
HIGHEST AVERAGE SUBJECT
Subject: English
Average: 70.94

LOWEST AVERAGE SUBJECT
Subject: Programming
Average: 62.14
```

Step 10: Save statistical output files

```
write.csv(
  subject_statistics,
  "descriptive_statistics.csv",
  row.names = FALSE
)

write.csv(
  section_performance,
  "section_performance.csv",
  row.names = FALSE
)

write.csv(
  section_subject_averages,
  "section_subject_averages.csv",
  row.names = FALSE
)
```

Step 11: Completion message

(Step 11 was cut off in the screenshot.)

Code 4: Data Visualization

Examination Result Analytics

Step 1: Load the prepared dataset

```
data <- read.csv(file.choose())
```

Step 2: Define subject columns

```
subjects <- c("Maths", "Physics", "Programming", "English")
```

Step 3: Calculate subject-wise average marks

```
subject_means <- colMeans(data[subjects], na.rm = TRUE)
```

Step 4: Create a bar chart of subject averages

```
png("01_subject_average.png", width = 900, height = 600)

barplot(
  subject_means,
  main = "Average Marks by Subject",
  xlab = "Subjects",
  ylab = "Average Marks",
  col = "skyblue",
  ylim = c(0, 100)
)

dev.off()
```

```
Output:
null device
          1
```

Step 5: Create section-wise performance chart

```
section_means <- aggregate(
  Percentage ~ Section,
  data = data,
  FUN = mean,
  na.action = na.omit
)

png("02_section_performance.png", width = 900, height = 600)

barplot(
  section_means$Percentage,
  names.arg = section_means$Section,
  main = "Average Percentage by Section",
  xlab = "Section",
  ...
```

(Rest of Step 5 and Step 6 were not visible in the screenshots.)

```
dev.off()
```

```
Output:
null device
          1
```

Step 7: Create grade distribution chart

```
grade_counts <- table(data$Grade)

png("04_grade_distribution.png", width = 900, height = 600)

barplot(
  grade_counts,
  main = "Student Grade Distribution",
  xlab = "Grade",
  ylab = "Number of Students",
  col = "orange"
)

dev.off()
```

```
Output:
null device
          1
```

Step 8: Display graph data in Console

```
cat("SUBJECT-WISE AVERAGE MARKS\n")
print(round(subject_means, 2))

cat("\nSECTION-WISE AVERAGE PERCENTAGE\n")
```

```
print(section_means)

cat("\nGRADE DISTRIBUTION\n")
print(grade_counts)
```

Output:

```
SUBJECT-WISE AVERAGE MARKS
      Maths     Physics Programming     English
      66.49       66.78       62.14       70.94

SECTION-WISE AVERAGE PERCENTAGE
  Section Percentage
1       A   65.97833
2       B   67.19500

GRADE DISTRIBUTION

Distinction       First        Pass      Second
          4          46           3           7
```

Code 5: Grade Analysis and Student Performance

Examination Result Analytics

Step 1: Load the prepared dataset

```
data <- read.csv(file.choose())
```

Step 2: Define subject columns

```
subjects <- c("Maths", "Physics", "Programming", "English")
```

Step 3: Calculate total marks and percentage

```
data$Total <- rowSums(data[, subjects], na.rm = FALSE)
data$Percentage <- data$Total / length(subjects)
```

Step 4: Assign grades

```
data$Grade <- cut(
  data$Percentage,
  breaks = c(-Inf, 50, 60, 75, Inf),
  labels = c("Pass", "Second", "First", "Distinction"),
  right = FALSE
)
```

Step 5: Display grade-wise student count

```
cat("GRADE-WISE STUDENT COUNT\n")
grade_counts <- table(data$Grade, useNA = "ifany")
print(grade_counts)
```

```
Output:
GRADE-WISE STUDENT COUNT

       Pass      Second       First Distinction
          3           7          46           4
```

Step 6: Calculate pass and fail count

```
cat("\nPASS AND FAIL SUMMARY\n")

pass_count <- sum(data$Percentage >= 50, na.rm = TRUE)
fail_count <- sum(data$Percentage < 50, na.rm = TRUE)

cat("Total Students:", nrow(data), "\n")
cat("Passed Students:", pass_count, "\n")
cat("Failed Students:", fail_count, "\n")
cat(
  "Pass Percentage:",
  round(pass_count / nrow(data) * 100, 2),
  "%\n"
)
```

```
Output:
PASS AND FAIL SUMMARY

Total Students: 60
Passed Students: 57
Failed Students: 3
Pass Percentage: 95 %
```

Step 7: Identify highest scoring student

```
cat("\nHIGHEST SCORING STUDENT\n")

highest_index <- which.max(data$Percentage)

print(data[highest_index, c(
  "Roll_No", "Section", "Total", "Percentage", "Grade"
)])
```

```
Output:
HIGHEST SCORING STUDENT

   Roll_No Section Total Percentage       Grade
43     143       A 315.8      78.95 Distinction
```

Step 8: Identify lowest scoring student

```
cat("\nLOWEST SCORING STUDENT\n")

lowest_index <- which.min(data$Percentage)

print(data[lowest_index, c(
  "Roll_No", "Section", "Total", "Percentage", "Grade"
)])
```

```
Output:
LOWEST SCORING STUDENT

  Roll_No Section Total Percentage Grade
1     101       A 154.8       38.7  Pass
```

Step 9: Display top 5 students

```
cat("\nTOP 5 STUDENTS\n")

top_students <- data[
  order(data$Percentage, decreasing = TRUE),
  c("Roll_No", "Section", "Total", "Percentage", "Grade")
]

top_students <- head(top_students, 5)
print(top_students)
```

```
Output:
TOP 5 STUDENTS

   Roll_No Section Total Percentage       Grade
43     143       A 315.8     78.950 Distinction
39     139       B 301.5     75.375 Distinction
12     112       B 300.8     75.200 Distinction
34     134       B 300.0     75.000 Distinction
58     158       B 299.9     74.975       First
```

Step 10: Section-wise grade distribution

```
cat("\nSECTION-WISE GRADE DISTRIBUTION\n")
print(table(data$Section, data$Grade, useNA = "ifany"))
```

```
Output:
SECTION-WISE GRADE DISTRIBUTION

   Pass Second First Distinction
  A    2      4    23           1
  B    1      3    23           3
```

Step 11: Save grade analysis results

```
write.csv(
  as.data.frame(grade_counts),
  "grade_distribution.csv",
  row.names = FALSE
)

write.csv(
  top_students,
  "top_5_students.csv",
  row.names = FALSE
)

write.csv(
  data,
  "student_performance_analysis.csv",
  row.names = FALSE
)
```

Step 12: Completion message

```
cat("\nGRADE ANALYSIS COMPLETED SUCCESSFULLY\n")
cat("Output files have been saved.\n")
```

```
Output:
GRADE ANALYSIS COMPLETED SUCCESSFULLY
Output files have been saved.
```

Code 6: Outlier Detection and Visualization

Examination Result Analytics

Step 1: Load the prepared dataset

```
data <- read.csv(file.choose())
```

Step 2: Calculate total marks and percentage

```
subjects <- c("Maths", "Physics", "Programming", "English")

data$Total <- rowSums(data[, subjects], na.rm = FALSE)
data$Percentage <- data$Total / length(subjects)
```

Step 3: Calculate quartiles and IQR

```
Q1 <- as.numeric(quantile(data$Total, 0.25, na.rm = TRUE))
Q3 <- as.numeric(quantile(data$Total, 0.75, na.rm = TRUE))
IQR_value <- IQR(data$Total, na.rm = TRUE)

lower_fence <- Q1 - 1.5 * IQR_value
upper_fence <- Q3 + 1.5 * IQR_value

cat("OUTLIER DETECTION USING IQR METHOD\n")
cat("Q1:", round(Q1, 2), "\n")
cat("Q3:", round(Q3, 2), "\n")
cat("IQR:", round(IQR_value, 2), "\n")
cat("Lower Fence:", round(lower_fence, 2), "\n")
cat("Upper Fence:", round(upper_fence, 2), "\n")
```

```
Output:
OUTLIER DETECTION USING IQR METHOD
Q1: 251.2
Q3: 292.32
IQR: 41.12
Lower Fence: 189.51
Upper Fence: 354.01
```

Step 4: Identify outliers

```
outlier_flag <- data$Total < lower_fence |
  data$Total > upper_fence

outliers <- data[
  outlier_flag,
  c("Roll_No", "Section", "Total", "Percentage")
]
```

(Remaining steps - outlier display and file saving - were not visible in the screenshots.)

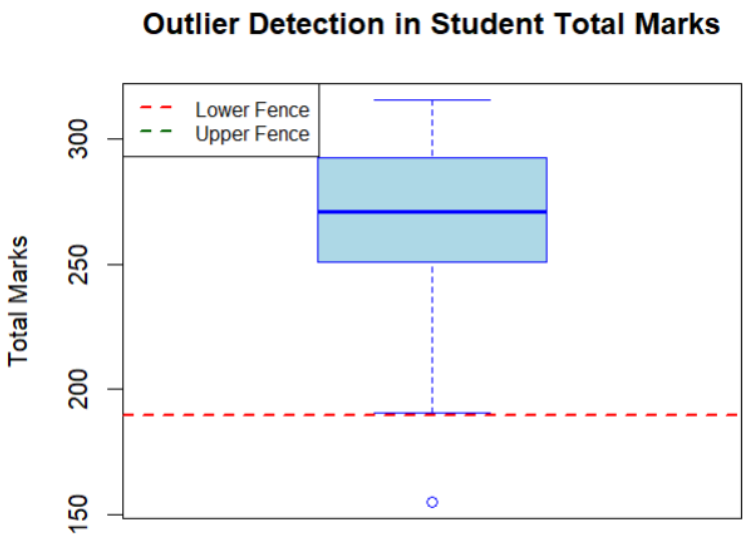
Boxplot with fence lines

```
boxplot(
  data$Total,
  main = "Outlier Detection in Student Total Marks",
  ylab = "Total Marks",
  col = "lightblue",
  border = "blue"
)

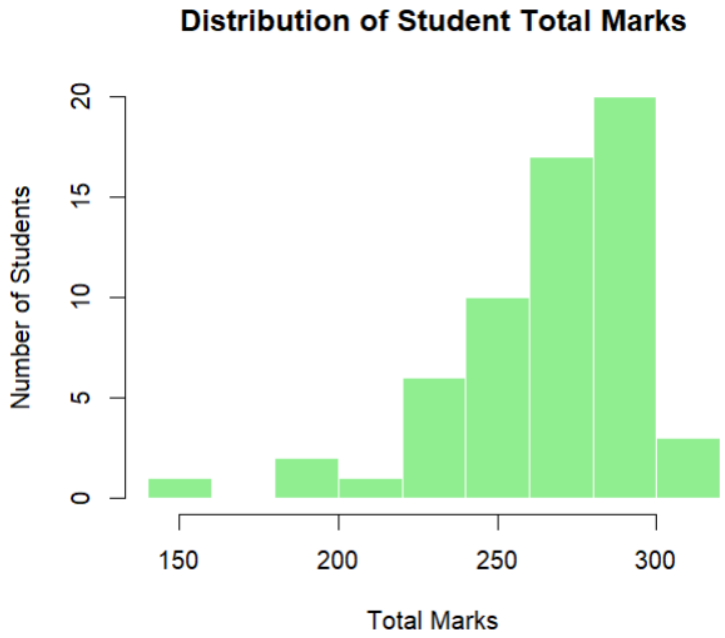
# Add lower and upper fence lines
abline(h = lower_fence, col = "red", lty = 2, lwd = 2)
abline(h = upper_fence, col = "darkgreen", lty = 2, lwd = 2)

legend(
  "topleft",
  legend = c("Lower Fence", "Upper Fence"),
  col = c("red", "darkgreen"),
  lty = 2,
  lwd = 2,
  cex = 0.8
)
```

```
Output:
OUTLIER ANALYSIS COMPLETED SUCCESSFULLY
Boxplot and histogram have been saved as PNG files.
```



Plot: Outlier Detection in Student Total Marks (Boxplot)



Plot: Distribution of Student Total Marks (Histogram)

Code 7: Correlation Analysis and Heatmap

Examination Result Analytics

Step 1: Load the prepared dataset

```
data <- read.csv(file.choose())
```

Step 2: Select subject columns

```
subjects <- c("Maths", "Physics", "Programming", "English")

marks_data <- data[, subjects]
```

Step 3: Calculate correlation matrix

```
correlation_matrix <- cor(
  marks_data,
  use = "pairwise.complete.obs",
  method = "pearson"
)

cat("CORRELATION MATRIX\n")
print(round(correlation_matrix, 2))
```

```
Output:
CORRELATION MATRIX
             Maths Physics Programming English
Maths         1.00    0.08       -0.09    0.20
Physics       0.08    1.00        0.00    0.01
Programming  -0.09    0.00        1.00    0.27
English       0.20    0.01        0.27    1.00
```

Step 4: Save correlation matrix

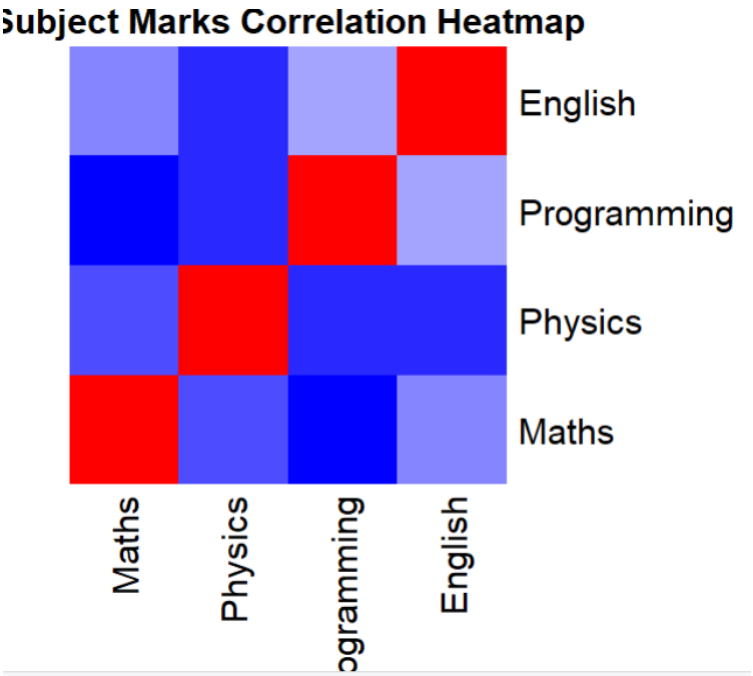
```
write.csv(
  round(correlation_matrix, 3),
  "correlation_matrix.csv"
)
```

Step 5: Display correlation heatmap in RStudio

```
heat_colors <- colorRampPalette(
  c("blue", "white", "red")
)(100)

heatmap(
  correlation_matrix,
  Rowv = NA,
  Colv = NA,
  scale = "none",
  ...
```

(Rest of the heatmap() arguments were cut off in the screenshot.)



Plot: Subject Marks Correlation Heatmap